

MID TERM EXAMINATION

Subject: datastructures

Paper: Mid Term Examination - Set A | Duration: 3 Hours | Max Marks: 104 | Date: 13-03-2026

Total Questions: 26 | Answer all questions.

Name: _____

Roll No: _____

Student Sign: _____

Teacher Sign: _____

SECTION A: MCQS

- Here are 10 unique university-level questions based on the provided metadata: --- EXAM: Mid Term Examination | Course: Data Structures (CS101) | Semester: 3 | Set A --- [2 Marks]
- Which of the following best describes the time complexity of searching an element in a sorted array using binary search? a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$ d) $O(1)$ [2 Marks]
- An Abstract Data Type (ADT) is primarily characterized by: a) Its specific implementation details using a programming language. b) The set of operations defined on the data and their behavior, independent of implementation. c) The amount of memory it consumes at runtime. d) Its ability to directly access memory addresses. [2 Marks]
- In a Last-In, First-Out (LIFO) data structure, which operation is primarily responsible for adding an element to the collection? a) Dequeue b) Pop c) Peek d) Push [2 Marks]
- Read 5: Stack: [5] [2 Marks]
- Read +: Pop 3, Pop 5. Compute $5 + 3 = 8$. Push 8. Stack: [8] [2 Marks]
- Read 2: Stack: [8, 8, 2] [2 Marks]
- Read : Pop 4, Pop 8. Compute $8 - 4 = 32$. Push 32. Stack: [32] [2 Marks]
- A significant advantage of a circular queue over a linear queue implemented using an array is that it: a) Allows elements to be accessed randomly. b) Eliminates the problem of unutilized space when elements are dequeued from the front. c) Provides a constant time complexity for searching elements. d) Automatically sorts the elements upon insertion. [2 Marks]
- When implementing a linked list, memory for each node is typically allocated dynamically. Which C++ keyword is used for dynamic memory allocation for objects or structs? a) static b) delete c) new d) malloc [2 Marks]
- Describe the algorithm and pseudo-code for inserting a new node with data X at a specific position pos (1-indexed) in a singly linked list. Assume head is the pointer to the first node. Handle edge cases like inserting at the beginning or if the position is out of bounds. Algorithm: [2 Marks]

12. Handle insertion at the beginning (pos == 1): Set the next pointer of the new node to point to the current head. Update head to point to the new node. **[2 Marks]**

SECTION B: SHORT ANSWERS

13. Briefly distinguish between a "data structure" and an "algorithm" in the context of computer science. A data structure is a particular way of organizing and storing data in a computer so that it can be accessed and modified efficiently. It defines the logical relationship between data items and how they are stored (e.g., array, linked list, tree). An algorithm is a step-by-step procedure or a set of rules used to solve a specific problem or perform a computation. It operates on data, often using specific data structures, to achieve a desired outcome (e.g., sorting algorithm, search algorithm). In essence, data structures organize data, while algorithms process that organized data. **[4 Marks]**

14. Evaluate the following postfix expression using a stack: $5\ 3\ +\ 8\ 2\ /\ *\ 7\ -$ Show the state of the stack after each operation. **[5 Marks]**

15. Read 3: Stack: [5, 3] **[5 Marks]**

16. Read 8: Stack: [8, 8] **[5 Marks]**

17. Read /: Pop 2, Pop 8. Compute $8 / 2 = 4$. Push 4. Stack: [8, 4] **[5 Marks]**

18. Read 7: Stack: [32, 7] **[5 Marks]**

19. Bidirectional Traversal: A doubly linked list can be traversed in both forward and backward directions, which is not possible with a singly linked list. This simplifies operations that require moving to the previous node (e.g., deleting a node given only a pointer to it, without knowing its predecessor). **[5 Marks]**

20. Create a new node: Allocate memory for the new node and set its data to X. **[5 Marks]**

21. Handle insertion at other positions (pos > 1): Initialize a temp pointer to head and a count to 1. Traverse the list until temp points to the node before the desired insertion position, i.e., count reaches pos - 1. During traversal, if temp becomes NULL before reaching pos - 1, it means the position is out of bounds. Print an error and return. Once temp is at the correct preceding node: Set the next pointer of the new node to temp->next. Set temp->next to point to the new node. **[5 Marks]**

SECTION C: DESCRIPTIVES

22. Explain two primary advantages of using a Doubly Linked List over a Singly Linked List. Provide a scenario where a Doubly Linked List would be preferred. Advantages: **[5 Marks]**

SECTION D: MCQS

23. Efficient Deletion (without predecessor): Deleting a specific node in a doubly linked list is more efficient ($O(1)$ after finding the node) because the node itself contains a pointer to its predecessor, allowing the pointers of adjacent nodes to be easily updated. In a singly linked list, deleting a node requires knowing its predecessor, often necessitating a

traversal from the head ($O(n)$) to find it. Scenario: A music playlist application where users can move to the "previous" or "next" song. A Doubly Linked List is ideal here because each song (node) can easily point to the song played before it and the song to be played next, facilitating quick navigation in both directions. **[6 Marks]**

SECTION E: DESCRIPTIVES

24. Compare and contrast static arrays with singly linked lists based on memory allocation strategy, insertion/deletion efficiency, and random access capability. Memory Allocation: Static Arrays: Contiguous memory block, size fixed at compile-time or initialization. Memory allocated on stack or heap. Singly Linked Lists: Non-contiguous memory allocation, nodes are allocated dynamically on the heap as needed. Insertion/Deletion Efficiency: Static Arrays: Inefficient ($O(n)$) for insertions/deletions in the middle, as elements need to be shifted. Efficient ($O(1)$) at the end if space is available. Singly Linked Lists: Efficient ($O(1)$) for insertions/deletions at the beginning or end (with tail pointer), and $O(n)$ for specific positions (requires traversal). No element shifting. Random Access Capability: Static Arrays: Efficient ($O(1)$) due to contiguous memory and direct indexing. * Singly Linked Lists: Inefficient ($O(n)$) as it requires sequential traversal from the head. **[7 Marks]**

SECTION F: MCQS

25. Read -: Pop 7, Pop 32. Compute $32 - 7 = 25$. Push 25. Stack: [25] Final Result: 25 **[8 Marks]**

26. Handle empty list: If the list is empty and $pos == 1$, the new node becomes the head. If $pos > 1$ and the list is empty, it's an invalid position. Pseudo-code: Function `insertAtPosition(head, X, pos)`: Create `newNode` `newNode.data = X` `newNode.next = NULL` If $pos == 1$: `newNode.next = head` `head = newNode` Return `head` `temp = head` `count = 1` While `temp is not NULL AND count < pos - 1`: `temp = temp.next` `count = count + 1` If `temp is NULL`: // Position out of bounds Print "Error: Position out of bounds." Return `head` // Or handle error appropriately // Insert `newNode` between `temp` and `temp.next` `newNode.next = temp.next` `temp.next = newNode` Return `head` **[10 Marks]**

--- End of Question Paper ---